ACADEMIC TASK NO:	CA 3
NAME:	**Rohit Kumar**
REGISTRATION NUMBER:	**12516608**
SECTION:	**D1P2501**
FACULTY NAME:	**Yamini Bhardwaj**

Q1. Running Sum of 1D Array:-

```
class Solution {
    public int[] runningSum(int[] nums) {

        for (int i = 1; i < nums.length; i++) {
            nums[i] = nums[i] + nums[i - 1];
        }

        return nums;
    }
}
```

Q2. 1672. Richest Customer Wealth:-

```
class Solution {
    public int maximumWealth(int[][] accounts) {

        int maxWealth = 0;

        for (int i = 0; i < accounts.length; i++) {
            int sum = 0;

            for (int j = 0; j < accounts[i].length; j++) {
```

```
            sum += accounts[i][j];
        }

        maxWealth = Math.max(maxWealth, sum);
    }

    return maxWealth;
  }
}
```

Q3. 1295. Find Numbers with Even Number of Digits:-

```
class Solution {
  public int findNumbers(int[] nums) {

    int count = 0;

    for (int num : nums) {
      int digits = 0;

      while (num > 0) {
        digits++;
        num /= 10;
      }

      if (digits % 2 == 0) {
        count++;
```

```
        }
      }

      return count;
   }
}
```

Q4 1929. Concatenation of Array

```
class Solution {
   public int[] getConcatenation(int[] nums) {

      int n = nums.length;
      int[] ans = new int[2 * n];

      for (int i = 0; i < n; i++) {
         ans[i] = nums[i];
         ans[i + n] = nums[i];
      }

      return ans;
   }
}
```

Q5 1603. Design Parking System

```
class ParkingSystem {
```

```
int big;
int medium;
int small;

public ParkingSystem(int big, int medium, int small) {
    this.big = big;
    this.medium = medium;
    this.small = small;
}

public boolean addCar(int carType) {

    if (carType == 1 && big > 0) {
        big--;
        return true;
    }

    if (carType == 2 && medium > 0) {
        medium--;
        return true;
    }

    if (carType == 3 && small > 0) {
        small--;
        return true;
    }

    return false;
```

```
    }
}
```

Q6 509. Fibonacci Number

```
class Solution {
    public int fib(int n) {

        if (n <= 1) {
            return n;
        }

        int a = 0;
        int b = 1;

        for (int i = 2; i <= n; i++) {
            int c = a + b;
            a = b;
            b = c;
        }

        return b;
    }
}
```

Q7 258. Add Digits

```
class Solution {
    public int addDigits(int num) {
```

```
        while (num >= 10) {
            int sum = 0;

            while (num > 0) {
                sum += num % 10;
                num /= 10;
            }

            num = sum;
        }

        return num;
    }
}
```

Q8 231. Power of Two

```
class Solution {
    public boolean isPowerOfTwo(int n) {

        if (n <= 0) {
            return false;
        }

        return (n & (n - 1)) == 0;
    }
}
```

Q9 1281. Subtract the Product and Sum of Digits of an Integer

```
class Solution {
    public int subtractProductAndSum(int n) {

        int sum = 0;
        int product = 1;

        while (n > 0) {
            int digit = n % 10;

            sum += digit;
            product *= digit;

            n /= 10;
        }

        return product - sum;
    }
}
```

Q10 75. Sort Colors

```
class Solution {
    public void sortColors(int[] nums) {

        int low = 0;
        int mid = 0;
        int high = nums.length - 1;
```

```
        while (mid <= high) {

            if (nums[mid] == 0) {
                int temp = nums[low];
                nums[low] = nums[mid];
                nums[mid] = temp;

                low++;
                mid++;
            }

            else if (nums[mid] == 1) {
                mid++;
            }

            else {
                int temp = nums[mid];
                nums[mid] = nums[high];
                nums[high] = temp;

                high--;
            }
        }
    }
}
```

Q11 151. Reverse Words in a String

```
class Solution {
    public String reverseWords(String s) {

        String[] words = s.trim().split("\\s+");

        StringBuilder result = new StringBuilder();

        for (int i = words.length - 1; i >= 0; i--) {

            result.append(words[i]);

            if (i > 0) {
                result.append(" ");
            }
        }

        return result.toString();
    }
}
```

Q12 7. Reverse Integer

```
class Solution {
    public int reverse(int x) {

        int result = 0;
```

```
        while (x != 0) {

            int digit = x % 10;
            x /= 10;

            if (result > Integer.MAX_VALUE / 10 ||
                (result == Integer.MAX_VALUE / 10 && digit > 7)) {
                return 0;
            }

            if (result < Integer.MIN_VALUE / 10 ||
                (result == Integer.MIN_VALUE / 10 && digit < -8)) {
                return 0;
            }

            result = result * 10 + digit;
        }

        return result;
    }
}
```

Q13 50. Pow(x, n)

```
class Solution {
    public double myPow(double x, int n) {

        return Math.pow(x, n);
```

```
    }
}
```

Q14 5. Longest Palindromic Substring

```
class Solution {

    public String longestPalindrome(String s) {

        if (s == null || s.length() < 2) {
            return s;
        }

        int start = 0;
        int end = 0;

        for (int i = 0; i < s.length(); i++) {

            int len1 = expand(s, i, i);
            int len2 = expand(s, i, i + 1);

            int len = Math.max(len1, len2);

            if (len > end - start + 1) {

                start = i - (len - 1) / 2;
                end = i + len / 2;
```

```
            }
        }

        return s.substring(start, end + 1);
    }

    private int expand(String s, int left, int right) {

        while (left >= 0 && right < s.length()
                && s.charAt(left) == s.charAt(right)) {

            left--;
            right++;
        }

        return right - left - 1;
    }
}
```

Q15 42. Trapping Rain Water

```
class Solution {
    public int trap(int[] height) {

        int left = 0;
        int right = height.length - 1;

        int leftMax = 0;
        int rightMax = 0;
```

```
int water = 0;

while (left < right) {

   if (height[left] < height[right]) {

      if (height[left] >= leftMax) {
         leftMax = height[left];
      } else {
         water += leftMax - height[left];
      }

      left++;
   }

   else {

      if (height[right] >= rightMax) {
         rightMax = height[right];
      } else {
         water += rightMax - height[right];
      }

      right--;
   }
}
```

```
        return water;
    }
}
```

Q16 273. Integer to English Words

```
class Solution {

    private final String[] belowTwenty = {
        "", "One", "Two", "Three", "Four", "Five",
        "Six", "Seven", "Eight", "Nine", "Ten",
        "Eleven", "Twelve", "Thirteen", "Fourteen",
        "Fifteen", "Sixteen", "Seventeen", "Eighteen",
        "Nineteen"
    };

    private final String[] tens = {
        "", "", "Twenty", "Thirty", "Forty",
        "Fifty", "Sixty", "Seventy", "Eighty", "Ninety"
    };

    private final String[] thousands = {
        "", "Thousand", "Million", "Billion"
    };

    public String numberToWords(int num) {

        if (num == 0) {
            return "Zero";
```

```
    }

    String result = "";
    int group = 0;

    while (num > 0) {

        int part = num % 1000;

        if (part != 0) {

            String words = helper(part);

            if (!thousands[group].isEmpty()) {
                words += " " + thousands[group];
            }

            result = words + (result.isEmpty() ? "" : " " + result);
        }

        num /= 1000;
        group++;
    }

    return result;
}

private String helper(int num) {
```

```
        if (num == 0) {
            return "";
        }

        if (num < 20) {
            return belowTwenty[num];
        }

        if (num < 100) {
            return tens[num / 10] +
                (num % 10 == 0 ? "" : " " + helper(num % 10));
        }

        return belowTwenty[num / 100] + " Hundred" +
            (num % 100 == 0 ? "" : " " + helper(num % 100));
    }
}
```